DLP Lab3: MaskGIT for Image Inpainting

Yi-Hsiang Ho, 111550106

1. Introduction

In this lab, we implement the Masked Generative Image Transformer (MaskGIT) for image inpainting. MaskGIT utilizes a bidirectional transformer and non-autoregressive iterative decoding, enabling parallel token prediction for faster and more coherent image synthesis compared to traditional sequential models.

We implement the following components:

- Multi-Head Self-Attention: Developed the transformer module for global context capture.
- Stage 2 Training: Trained the transformer to predict masked image tokens.
- Iterative Decoding: Implemented inpainting logic with confidence-based sampling.
- Mask-Scheduling: Integrated dynamic scheduling to manage mask ratios across iterations.

Results were evaluated using the Fréchet Inception Distance (FID) to quantify image quality and diversity.

2. Implementation Details

2.1. Multi-Head Self-Attention

The multi-head attention module uses the scaled dot-product mechanism to model global relationships across 16 parallel heads. Input features are projected into query, key, and value tensors, then split for head-wise computation. Attention scores are computed as $\mathbf{QK}^T/\sqrt{d_k}$, and normalized via softmax. Then multiply with $\mathbf{V}$. The resulting context vectors are concatenated and passed through a final linear layer to restore the original embedding dimension while integrating diverse spatial information.

The implementation is as follows:

```python
class MultiHeadAttention(nn.Module):
    def __init__(self, dim=768, num_heads=16, attn_drop=0.1):
        super(MultiHeadAttention, self).__init__()
        self.dim = dim
        self.num_heads = num_heads
        self.head_dim = dim // num_heads
        self.scale = self.head_dim ** -0.5
        self.wq = nn.Linear(dim, dim)
        self.wk = nn.Linear(dim, dim)
        self.wv = nn.Linear(dim, dim)
        self.drop = nn.Dropout(attn_drop)
        self.wo = nn.Linear(dim, dim)

    def forward(self, x):
        ''' Hint: input x tensor shape is (batch_size, num_image_tokens, dim),
            because the bidirectional transformer first will embed each token to dim dimension,
            and then pass to n_layers of encoders consist of Multi-Head Attention and MLP.
            # of head set 16
            Total d_k , d_v set to 768
            d_k , d_v for one head will be 768//16.
        '''
        B, N, dim = x.shape
        q = self.wq(x)  # (B, N, dim)
        k = self.wk(x)
        v = self.wv(x)
        q = q.view(B, N, self.num_heads, self.head_dim).transpose(1, 2)  # (B, num_heads, N, head_dim)
        k = k.view(B, N, self.num_heads, self.head_dim).transpose(1, 2)
        v = v.view(B, N, self.num_heads, self.head_dim).transpose(1, 2)
        attn = torch.matmul(q, k.transpose(-2, -1)) * self.scale  # (B, num_heads, N, N)
        attn = F.softmax(attn, dim=-1)
        attn = self.drop(attn)
        attn = torch.matmul(attn, v)  # (B, num_heads, N, head_dim)
        attn = attn.transpose(1, 2).contiguous().view(B, N, dim)  # (B, N, dim)
        return self.wo(attn)
```

Figure 1. **The implementation of multi-head attention**

2.2. Stage 2 Training

Stage 2 training focuses on optimizing the bidirectional transformer through Masked Visual Token Modeling (MVTM). The input image is first encoded into a 16×16 grid of discrete visual tokens using a pre-trained and frozen VQGAN encoder (Fig. 2). During each training step (Fig. 3), a masking ratio is sampled based on a cosine scheduling function (Fig. 4), and a corresponding subset of tokens is randomly replaced with a special mask token ID. In the forward pass, these masked indices are fed into the bidirectional transformer to predict the original visual tokens. The model is trained using cross-entropy loss between the predicted logits and the ground-truth tokens, incorporating a label smoothing of 0.1 to improve robustness and prevent over-confident predictions. Additionally, gradient accumulation is implemented to simulate a larger effective batch size, where gradients are normalized and accumulated over multiple mini-batches before applying gradient clipping and performing an optimization step.

```python
@torch.no_grad()
def encode_to_z(self, x):
    zq, z_indices, _ = self.vqgan.encode(x)
    z_indices = z_indices.view(zq.shape[0], -1)
    return zq, z_indices
```

Figure 2. **The implementation of Stage 2 Training (Fed into pretrained VQGAN encoder)**

```python
def forward(self, x):
    zq, z_indices = self.encode_to_z(x)

    # Generate mask
    r = np.random.uniform(0, 1)
    mask_ratio = self.gamma_func(r, mode=self.gamma_type)
    v = torch.rand(z_indices.shape, device=z_indices.device)
    mask = (v < mask_ratio)

    # Prepare masked indices
    masked_indices = z_indices.clone()
    masked_indices[mask] = self.mask_token_id

    logits = self.transformer(masked_indices)

    return logits, z_indices
```

Figure 3. **The implementation of Stage 2 Training (Forward pass)**

```python
def gamma_func(self, ratio, mode="cosine"):
    """Generates a mask rate by scheduling mask functions R.

    Given a ratio in [0, 1), we generate a masking ratio from (0, 1].
    During training, the input ratio is uniformly sampled;
    during inference, the input ratio is based on the step number divided by the total iteration number: t/T.
    Based on experiements, we find that masking more in training helps.

    ratio:   The uniformly sampled ratio [0, 1) as input.
    Returns: The mask rate (float).

    """
    if mode == "linear":
        mask_ratio = 1.0 - ratio
    elif mode == "cosine":
        mask_ratio = np.cos(np.pi / 2 * ratio)
    elif mode == "square":
        mask_ratio = 1.0 - ratio ** 2
    else:
        raise NotImplementedError

    return np.clip(mask_ratio, 1e-6, 1.0)
```

Figure 4. **The implementation of Stage 2 Training (Gamma function)**

```python
def train_one_epoch(self, train_loader, epoch):
    self.model.train()
    total_loss = 0
    pbar = tqdm(train_loader)
    self.optim.zero_grad()
    for i, images in enumerate(pbar):
        images = images.to(self.args.device)
        logits, targets = self.model(images)
        loss = F.cross_entropy(logits.view(-1, logits.size(-1)), targets.view(-1), label_smoothing=0.1)

        # Gradient Accumulation
        loss = loss / self.args.accum_grad
        loss.backward()

        if (i + 1) % self.args.accum_grad == 0 or (i + 1) == len(train_loader):
            # Gradient Clipping
            torch.nn.utils.clip_grad_norm_(self.model.transformer.parameters(), 1.0)
            self.optim.step()
            self.optim.zero_grad()

        total_loss += loss.item() * self.args.accum_grad
        pbar.set_description(f"Epoch: {epoch}, Loss: {loss.item() * self.args.accum_grad:.4f}")

        self.writer.add_scalar('Loss/train', loss.item() * self.args.accum_grad, epoch * len(train_loader) + i)
        self.writer.add_scalar('Learning Rate', self.scheduler.get_last_lr()[0], epoch * len(train_loader) + i)

    self.scheduler.step()

    avg_loss = total_loss / len(train_loader)
    return avg_loss
```

```python
def eval_one_epoch(self, val_loader, epoch):
    self.model.eval()
    total_loss = 0
    with torch.no_grad():
        pbar = tqdm(val_loader)
        for i, images in enumerate(pbar):
            images = images.to(self.args.device)
            logits, targets = self.model(images)
            loss = F.cross_entropy(logits.view(-1, logits.size(-1)), targets.view(-1))
            total_loss += loss.item()
            pbar.set_description(f"Eval Epoch: {epoch}, Loss: {loss.item():.4f}")

    avg_loss = total_loss / len(val_loader)
    self.writer.add_scalar('Loss/val', avg_loss, epoch)
    return avg_loss
```

```python
def configure_optimizers(self):
    optimizer = torch.optim.Adam(
        self.model.transformer.parameters(),
        lr=self.args.learning_rate,
        betas=(0.9, 0.96)
    )
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
        optimizer,
        T_max=self.args.epochs,
        eta_min=1e-6
    )
    return optimizer, scheduler
```

Figure 5. **The implementation of Stage 2 Training (Training)**

```python
best_val_loss = float('inf')
for epoch in range(args.start_from_epoch + 1, args.epochs + 1):
    train_loss = train_transformer.train_one_epoch(train_loader, epoch)
    val_loss = train_transformer.eval_one_epoch(val_loader, epoch)

    # Visualization
    train_transformer.visualize_inpainting(val_loader, epoch)

    if epoch % args.save_per_epoch == 0:
        torch.save(
            train_transformer.model.transformer.state_dict(),
            os.path.join("transformer_checkpoints", args.model_name, f"epoch_{epoch}.pt")
        )

    if val_loss < best_val_loss:
        best_val_loss = val_loss
        torch.save(
            train_transformer.model.transformer.state_dict(),
            os.path.join("transformer_checkpoints", args.model_name, f"best_val_loss.pt")
        )
train_transformer.writer.close()
```

Figure 6. **The implementation of Stage 2 Training (Training loop)**

2.3. Inference

The inference process in MaskGIT employs a non-autoregressive iterative decoding strategy to reconstruct the masked regions of an image. Starting from an initial set of masked visual tokens, the model performs a fixed number of decoding iterations T. In each iteration, the bidirectional transformer predicts the probability distributions for all masked positions in parallel. For each position, a token is sampled from the distribution, and a confidence score is assigned based on its prediction probability. To introduce diversity, temperature annealing and Gumbel noise are incorporated into the confidence calculation. A mask scheduling function $\gamma(t/T)$ determines the number of tokens to remain masked in the next step, where tokens with the lowest confidence scores are re-masked. This iterative refinement allows the model to progressively generate a coherent and high-quality image by focusing on the most certain regions first.

The implementation of the iterative decoding logic is shown below:

```python
def inpainting(self, z_indices, mask, ratio, initial_mask=None):
    logits = self.transformer(z_indices)  # (B, num_tokens, codebook_size+1)
    logits_for_prob = logits[:, :, :self.mask_token_id]  # ignore the last token (mask token)

    # Sample tokens instead of argmax
    probs = F.softmax(logits_for_prob, dim=-1)
    B, N, K = probs.shape
    z_indices_predict = torch.multinomial(probs.view(-1, K), 1).view(B, N)

    # Use the probability of the sampled token as confidence (add gumbel noise)
    z_indices_predict_prob = torch.gather(probs, -1, z_indices_predict.unsqueeze(-1)).squeeze(-1)
    temperature = self.choice_temperature * (1 - ratio)
    g = torch.distributions.Gumbel(0, 1).sample(z_indices_predict_prob.shape).to(z_indices.device)
    confidence = z_indices_predict_prob + temperature * g

    confidence[~mask] = float('inf')  # Unmasked region

    sorted_indices = torch.argsort(confidence, dim=-1)  # (B, num_tokens) ascending

    # How many tokens to unmask
    mask_ratio = self.gamma_func(ratio, mode=self.gamma_type)
    n_mask = math.ceil(mask_ratio * initial_mask.sum())

    # New mask: lowest confidence tokens stay masked
    mask_bc = torch.zeros_like(mask)
    for i in range(z_indices.shape[0]):
        mask_bc[i, sorted_indices[i, :n_mask]] = True

    # Ensure mask_bc is a subset of the original mask
    mask_bc = mask_bc & initial_mask

    # Update z_indices for the next step
    z_indices_new = z_indices.clone()
    # Revealed tokens: the ones that were masked but are now unmasked
    revealed = mask & ~mask_bc
    z_indices_new[revealed] = z_indices_predict[revealed]

    return z_indices_new, mask_bc, z_indices_predict
```

Figure 7. **The implementation of iterative decoding for inference (1 step)**

```python
def inpainting(self,image,mask_b,i): #MakGIT inference
    maska = torch.zeros(self.total_iter, 3, 16, 16) #save all iterations of masks in latent domain
    imga = torch.zeros(self.total_iter+1, 3, 64, 64)#save all iterations of decoded images
    mean = torch.tensor([0.4868, 0.4341, 0.3844],device=self.device).view(3, 1, 1)
    std = torch.tensor([0.2620, 0.2527, 0.2543],device=self.device).view(3, 1, 1)
    ori=(image[0]*std)+mean
    imga[0]=ori #mask the first image be the ground truth of masked image

    self.model.eval()
    with torch.no_grad():
        _, z_indices = self.model.encode_to_z(image)
        # Save original indices for ground truth preservation
        z_indices_gt = z_indices.clone()
        z_indices[mask_b] = self.model.mask_token_id

        mask_bc = mask_b.clone()
        initial_mask = mask_b.clone()

        for step in range(self.total_iter):
            ratio = (step + 1) / self.total_iter
            z_indices, mask_bc, z_indices_predict = self.model.inpainting(
                z_indices, mask_bc, ratio, initial_mask=initial_mask
            )

            # visualization: combine ground truth with current predictions
            z_indices_show = z_indices_predict.clone()
            z_indices_show[~initial_mask] = z_indices_gt[~initial_mask]

            # static method yon can modify or not, make sure your visualization results are correct
            mask_i=mask_bc.view(1, 16, 16)
            mask_image = torch.ones(3, 16, 16)
            indices = torch.nonzero(mask_i, as_tuple=False)#label mask true
            mask_image[:, indices[:, 1], indices[:, 2]] = 0 #3,16,16
            maska[step]=mask_image
            shape=(1,16,16,256)
            z_q = self.model.vqgan.codebook.embedding(z_indices_show).view(shape)
            z_q = z_q.permute(0, 3, 1, 2)
            decoded_img=self.model.vqgan.decode(z_q)
            dec_img_ori=(decoded_img[0]*std)+mean
            imga[step+1]=dec_img_ori #get decoded image

            # Save each step's result to test_results_all
            step_dir = os.path.join("test_results_all", f"{args.mask_func}", f"step_{step + 1}")
            vutils.save_image(dec_img_ori, os.path.join(step_dir, f"image_{i:03d}.png"), nrow=1)

            if step + 1 == self.sweet_spot:
                # Decoded image of the sweet spot only
                vutils.save_image(dec_img_ori, os.path.join("test_results", f"{args.mask_func}", f"image_{i:03d}.png"), nrow=1)

        #demo score
        vutils.save_image(maska, os.path.join("mask_scheduling", f"{args.mask_func}", f"test_{i}.png"), nrow=10)
        vutils.save_image(imga, os.path.join("imga", f"{args.mask_func}", f"test_{i}.png"), nrow=7)
```

Figure 8. **The implementation of iterative decoding for inference (inpaint loop)**

3. Experimental Score

3.1. Settings

Setting	Value
Learning rate	10^{-4}
Epochs	200
Optimizer	Adam ($\beta_1 = 0.9, \beta_2 = 0.96$)
Scheduler	Cosine annealing
Batch size	16
Gradient accum. steps	10
Gradient clipping thres.	1.0
Label smoothing	0.1

3.2. Iterative Decoding

The iterative decoding process is visualized in Fig. 9 and Fig. 10. As shown in the figures, linear mode reduces mask region linearly, while concave funcion (cosine, square-root) reduces mask region slowly at the beginning and quickly at the end.

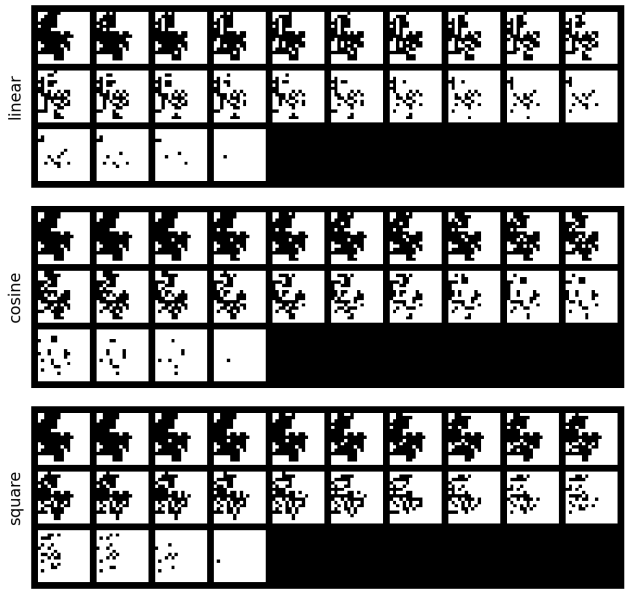


Figure 9. **Visualization of iterative decoding in latent space**



Figure 10. **Visualization of iterative decoding in pixel space**

3.3. FID Score

The model's performance was evaluated using the FID. My best FID score is 29.1414, as shown in Fig. 11. I also compared various mask scheduling functions, as shown in Fig. 12, where the cosine schedule consistently achieved the best results. The visual results of the best model are shown in Fig. 13.

```
(maskgit) yihsho@gpu2:/project2/yihsho/DLP/DLP_Lab3$ python faster-pytorch-fid/f
id_score_gpu.py --predicted-path test_results_all/cosine/step_24 --gtcsv-path fa
ster-pytorch-fid/test_gt.csv
747
100%|                                                  | 15/15 [00:00<00:00, 18.86it/s]
100%|                                                  | 15/15 [00:00<00:00, 30.78it/s]
FID:  29.14142346489575
```

Figure 11. **Screenshot of best FID score**

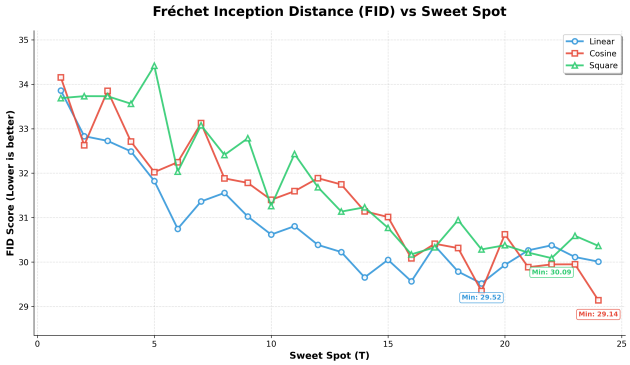


Figure 12. **FID score comparison across different mask functions**



Figure 13. **Generated images**

4. Discussion

4.1. Sampling vs. `argmax`

Although the initial skeleton code used `argmax` to select tokens, the original MaskGIT paper recommends sampling from the probability distribution. In my experiments, switching to stochastic sampling led to a better FID score.

4.2. Gradient Accumulation

Training transformers is often sensitive to batch size. However, GPU memory limits restricted the batch size to 16. By implementing gradient accumulation over 10 steps, I effectively simulated a batch size of 160. This larger effective batch size provided more stable gradient estimates during Stage 2 training, which led to smoother convergence and a noticeable improvement in the final FID score.